Docker-03 Assignment

Step-by-Step Implementation Guide

Objective

The objective of this assignment is to understand Docker image management, Dockerfile creation, Docker Compose, image security scanning, AWS ECR, and Jenkins pipeline automation.

Requirements

- Ubuntu EC2 instance with Docker installed
- SSH access to the instance
- Docker Hub account
- Git installed
- AWS account with ECR access
- Jenkins installed for the final pipeline task

First, check Docker installation.

docker --version
docker ps
git --version

```
root@ip-172-31-36-56:/home/ubuntu# docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
root@ip-172-31-36-56:/home/ubuntu# docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
root@ip-172-31-36-56:/home/ubuntu# git --version
git version 2.43.0
root@ip-172-31-36-56:/home/ubuntu#
```

Purpose:

- `docker --version` – Checks the installed Docker version.
- `docker ps` – Displays running containers.
- `git --version` – Checks whether Git is installed.

Task 1: Create an Image from a Running Container

Objective: Create a Docker image from an existing container using the `docker commit` command.

Step 1: Create and run a container

docker run -dit --name mycontainer ubuntu:latest

Explanation:

- `docker run` – Creates and starts a container.
- `-d` – Runs the container in the background.
- `-i` – Keeps standard input open.
- `-t` – Allocates a terminal.
- `--name mycontainer` – Assigns a name to the container.
- `ubuntu:latest` – Uses the Ubuntu base image.

Step 2: Check the running container

docker ps

This command displays the running container and its details.

```
root@ip-172-31-36-56:/home/ubuntu# docker run -dit --name mycontainer ubuntu:latest
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
8f70d2bfe91a: Download complete
Digest: sha256:f144425ff09be612d6d9ad965196e9cdc23dae1f42110a8a11a3e9a8198759f7
Status: Downloaded newer image for ubuntu:latest
d393f5a2c953969db585a6fe2cec2cca547d5a426dd68cded2a728a0b0235223
root@ip-172-31-36-56:/home/ubuntu# docker ps
CONTAINER ID   IMAGE           COMMAND       CREATED          STATUS          PORTS
    NAMES
d393f5a2c953   ubuntu:latest   "/bin/bash"   21 seconds ago   Up 21 seconds
    mycontainer
root@ip-172-31-36-56:/home/ubuntu#
```

Step 3: Make a change inside the container

docker exec -it mycontainer bash

Inside the container, run:

apt update
apt install -y curl
exit

This installs `curl` inside the container. The change can be captured in the new image.

Step 4: Create an image from the container

docker commit mycontainer myubuntu:v1

Explanation:

- `docker commit` – Creates a new image from a container's filesystem changes.
- `mycontainer` – The source container.
- `myubuntu:v1` – The new image name and tag.

Step 5: Verify the image

docker images

Expected result: The image `myubuntu` with tag `v1` should appear in the image list.



Conclusion: We created a new Docker image from a running container, including its filesystem changes.

Task 2: Copy an Image from the Local Machine to a Docker Server and Load It

Objective: Transfer a Docker image from your local computer to a remote Docker server without pushing it to a registry.

First Install Docker Desktop

Step 1: Download Docker Desktop

1. Open the official website: https://www.docker.com/products/docker-desktop/
2. Select **Download for Windows – x86_64**.

3. Download the `Docker Desktop Installer.exe` file.
4. Open the downloaded installer.

Step 2: Configure Installation

1. Select **All-users installation**.
2. Keep **Use WSL 2 instead of Hyper-V** checked.
3. Leave **Allow Windows Containers** unchecked.
4. Keep **Add shortcut to desktop** checked.
5. Click **OK**.

Step 3: Complete Installation

1. Wait until the installation is complete.
2. Restart the computer if prompted.
3. Open Docker Desktop and wait until it starts.

Step 4: Verify Installation

Open PowerShell and run:

docker --version

Output:

Docker version 29.8.2, build 7fc2dff

After that do this for task - 3

Step 1: Download an image on the local machine

docker pull nginx:latest

This downloads the Nginx image to your local Docker system.

```
PS C:\Users\chvka> docker pull nginx:latest
latest: Pulling from library/nginx
ecc510c1e359: Pull complete
947d22046304: Pull complete
f7e1db57b58b: Pull complete
8020170dc65f: Pull complete
1efc53192927: Pull complete
04bdbc7ea6b5: Pull complete
807ab930f3a0: Pull complete
a1b85ab28c90: Download complete
c329293b1d90: Download complete
Digest: sha256:f9ea18bfa4fad859e1ed38259d711da7ccad2c3516e875cec
3351a57c859f571
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
PS C:\Users\chvka>
```

Step 2: Save the image as a TAR file

docker save -o nginx-image.tar nginx:latest

Explanation:

- `docker save` – Saves the Docker image into an archive.
- `-o nginx-image.tar` – Specifies the output filename.
- `nginx:latest` – The image to save.

Step 3: Copy the image to the Docker server

Run this command from your local machine, replacing the placeholders with your server details:

scp -i "your-key.pem" nginx-image.tar ubuntu@YOUR_SERVER_IP:/home/ubuntu/

Explanation:

- `scp` – Securely copies files between computers.
- `-i "your-key.pem"` – Specifies the SSH private key.
- `ubuntu@YOUR_SERVER_IP` – Specifies the remote Ubuntu user and server IP.
- `/home/ubuntu/` – Destination directory.

If your local machine is Windows, run this command in PowerShell, provided OpenSSH is installed.

```
PS C:\Users\chvka> scp -i "$HOME\Downloads\docker-key.pem" "$HOME\Downloads\nginx-image.tar" ubuntu@98.81.130.38:/home/ubuntu/
C:\WINDOWS\System32\OpenSSH\scp.exe: stat local "C:/Users/chvka/Downloads/nginx-image.tar": No such file or directory
PS C:\Users\chvka> docker pull nginx:latest
latest: Pulling from library/nginx
Digest: sha256:f9ea18bfa4fad859e1ed38259d711da7ccad2c3516e875cec3351a57c859f571
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
```

Step 4: Connect to the Docker server

ssh -i "your-key.pem" ubuntu@YOUR_SERVER_IP

```
PS C:\Users\chvka> ssh -i "$HOME\Downloads\docker-key.pem" ubuntu@98.81.130.38
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-1014-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Oct 14 11:13:45 UTC 2026

  System load:  0.08              Temperature:           -273.1 C
  Usage of /:   72.0% of 6.71GB   Processes:             125
  Memory usage: 4%                Users logged in:       1
  Swap usage:   0%                IPv4 address for ens5: 172.31.36.56

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

43 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '26.04.1 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Wed Oct 14 10:32:16 2026 from 103.160.27.159
ubuntu@ip-172-31-36-56:~$
```

Step 5: Load the image on the server

Run on the remote Docker server:

docker load -i /home/ubuntu/nginx-image.tar

```
PS C:\Users\chvka> dir "$HOME\Downloads\nginx-image.tar"

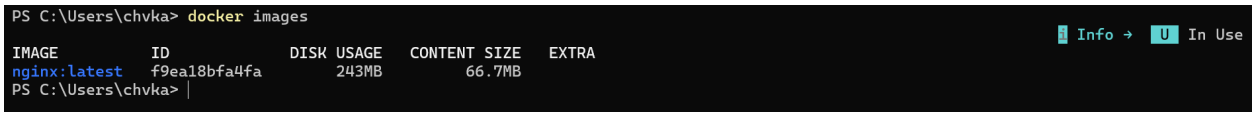

    Directory: C:\Users\chvka\Downloads


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        14-10-2026     16:45       66749440 nginx-image.tar
```

Step 6: Verify the image

docker images

Expected result: The `nginx` image with the `latest` tag should appear on the remote server.



Conclusion: We transferred a Docker image from a local machine to a remote server using `docker save`, `scp`, and `docker load`.

Task 3: Create a Docker Image Using Alpine and Customize It with Tomcat

Objective: Create a lightweight Docker image using Alpine Linux and install Tomcat inside it.

Step 1: Create a project directory

mkdir alpine-tomcat
cd alpine-tomcat

Step 2: Create a Dockerfile

nano Dockerfile

Add the following content:

FROM alpine:3.22

RUN apk add --no-cache openjdk17-jre curl tar

ENV CATALINA_HOME=/opt/tomcat
ENV PATH="$CATALINA_HOME/bin$:{PATH}"

RUN mkdir -p ${CATALINA_HOME} && \

```
    curl -fL https://archive.apache.org/dist/tomcat/tomcat-10/v10.1.44/bin/apache-tomcat-10.1.44.tar.gz \
    -o /tmp/tomcat.tar.gz && \
    tar -xzf /tmp/tomcat.tar.gz \
    --strip-components=1 -C ${CATALINA_HOME} && \
    rm /tmp/tomcat.tar.gz

EXPOSE 8080

CMD ["catalina.sh", "run"]
```

Save the file using `Ctrl+O`, press Enter, and exit with `Ctrl+X`.

Explanation:

- `FROM alpine:3.22` – Uses Alpine Linux as the base image.
- `RUN apk add` – Installs Java, curl, and tar.
- `ENV CATALINA_HOME` – Defines Tomcat's installation directory.
- `curl` – Downloads the Tomcat archive.
- `tar` – Extracts Tomcat into `/opt/tomcat`.
- `EXPOSE 8080` – Documents the port used by Tomcat.
- `CMD` – Starts Tomcat in the foreground.

This example uses a specific Tomcat release for reproducibility. For production, choose a currently supported, patched release and verify its download checksum.

Step 3: Build the Docker image

```
docker build -t alpine-tomcat:v1 .
```

Explanation:

- `docker build` – Builds an image from the Dockerfile.
- `-t alpine-tomcat:v1` – Assigns an image name and tag.
- `.` – Uses the current directory as the build context.

```
 ---> Running in 244fb7da11be
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 13.4M  100 13.4M    0     0  59018      0  0:03:59  0:03:59 --:--:-- 61253
 ---> Removed intermediate container 244fb7da11be
 ---> cb666920ca1f
Step 6/7 : EXPOSE 8080
 ---> Running in ec6aa2289927
 ---> Removed intermediate container ec6aa2289927
 ---> 818f13ac01b4
Step 7/7 : CMD ["catalina.sh", "run"]
 ---> Running in 19f5ea5987bc
 ---> Removed intermediate container 19f5ea5987bc
 ---> 9b13fdf855d3
Successfully built 9b13fdf855d3
Successfully tagged alpine-tomcat:v1
root@ip-172-31-36-56:/home/ubuntu/alpine-tomcat#
```

Step 4: Run the container

docker run -d --name tomcat-alpine -p 8083:8080 alpine-tomcat:v1

Step 5: Verify Tomcat

docker ps

```
root@ip-172-31-36-56:/home/ubuntu/alpine-tomcat# docker run -d --name tomact-alpine
-p 8083:8080 alpine-tomcat:v1
7d70f1528c803a6363c14660853037360 25e3b702a6c71d5b6a2f942d2e1e316
root@ip-172-31-36-56:/home/ubuntu/alpine-tomcat# docker ps
CONTAINER ID   IMAGE              COMMAND             CREATED             STATUS
         PORTS                                       NAMES
7d70f1528c80   alpine-tomcat:v1   "catalina.sh run"   6 seconds ago       Up 6 secon
ds       0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp   tomact-alpine
d393f5a2c953   ubuntu:latest      "/bin/bash"         About an hour ago   Up About a
n hour                                                 mycontainer
root@ip-172-31-36-56:/home/ubuntu/alpine-tomcat#
```

Before doing that Execute this commands

docker exec tomact-alpine sh -c 'mkdir -p /opt/tomcat/webapps/sample'

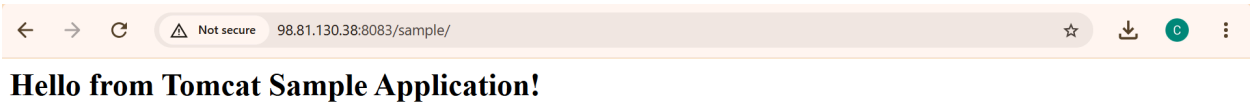
docker exec tomact-alpine sh -c 'echo "<html><body><h1>Hello from Tomcat Sample Application!</h1></body></html>" > /opt/tomcat/webapps/sample/index.html'

Then test the application from your computer or browser:

http://YOUR_SERVER_IP:8080

Ensure the EC2 security group allows inbound TCP traffic on port `8080` if you are testing from outside the server.

Conclusion: We created a customized Alpine-based Docker image with Java and Tomcat and ran it as a container.



Task 4: Create Single-Stage and Multi-Stage Dockerfiles

Objective: Build Docker images for the source code using both single-stage and multi-stage Dockerfiles and compare the results.

Source code: https://github.com/betawins/multi-stage-example.git

Step 1: Clone the source code

```
git clone https://github.com/betawins/multi-stage-example.git
cd multi-stage-example
ls -la
```

This downloads the source code and displays the project files. Check for `pom.xml`, `mvnw`, `.mvn`, and `src` before continuing, because the Dockerfiles below assume a Maven-based Java project.

```
root@ip-172-31-36-56:/home/ubuntu# git clone https://github.com/betawins/multi-stage-example.git
Cloning into 'multi-stage-example'...
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 31 (delta 2), reused 1 (delta 1), pack-reused 24 (from 1)
Receiving objects: 100% (31/31), 53.25 KiB | 10.65 MiB/s, done.
Resolving deltas: 100% (3/3), done.
root@ip-172-31-36-56:/home/ubuntu# cd multi-stage-example
root@ip-172-31-36-56:/home/ubuntu/multi-stage-example# ls -la
total 56
drwxr-xr-x 5 root   root    4096 Oct 14 11:52 .
drwxr-x--- 6 ubuntu ubuntu  4096 Oct 14 11:52 ..
drwxr-xr-x 8 root   root    4096 Oct 14 11:52 .git
-rw-r--r-- 1 root   root     333 Oct 14 11:52 .gitignore
drwxr-xr-x 3 root   root    4096 Oct 14 11:52 .mvn
-rw-r--r-- 1 root   root     297 Oct 14 11:52 Dockerfile
-rw-r--r-- 1 root   root     349 Oct 14 11:52 README.md
-rwxr-xr-x 1 root   root   10070 Oct 14 11:52 mvnw
-rw-r--r-- 1 root   root    6608 Oct 14 11:52 mvnw.cmd
-rw-r--r-- 1 root   root    1565 Oct 14 11:52 pom.xml
drwxr-xr-x 4 root   root    4096 Oct 14 11:52 src
root@ip-172-31-36-56:/home/ubuntu/multi-stage-example#
```

Step 2: Create a single-stage Dockerfile

Create a file named `Dockerfile.single`:

nano Dockerfile.single

Add:

```
FROM maven:3.9-eclipse-temurin-21

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

EXPOSE 8080

CMD ["sh", "-c", "java -jar target/*.jar"]
```

Explanation:

- `FROM` – Uses an image containing Maven and Java.
- `WORKDIR` – Sets the working directory inside the image.
- `COPY` – Copies the project source code.
- `RUN` – Builds the application and generates a JAR file.

- `EXPOSE` – Documents the application port.
- `CMD` – Starts the packaged Java application.

This approach uses a build environment for the application and retains the build tools and source files in the resulting image.

Step 3: Build the single-stage image

docker build -f Dockerfile.single -t multi-stage-example:single .

```
 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-settings/3.3.9/maven-settings-3.3.9.jar (44 kB at 216 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-artifact-transfer/0.12.0/maven-artifact-transfer-0.12.0.jar
Downloading from central: https://repo.maven.apache.org/maven2/commons-codec/commons-codec/1.13/commons-codec-1.13.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/3.7.0/plexus-archiver-3.7.0.jar (192 kB at 891 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/7.3.1/asm-7.3.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.1.0/plexus-io-3.1.0.jar (75 kB at 347 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-commons/7.3.1/asm-commons-7.3.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-artifact-transfer/0.12.0/maven-artifact-transfer-0.12.0.jar (120 kB at 508 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-tree/7.3.1/asm-tree-7.3.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-shade-plugin/3.2.2/maven-shade-plugin-3.2.2.jar (131 kB at 524 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-analysis/7.3.1/asm-analysis-7.3.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-commons/7.3.1/asm-commons-7.3.1.jar (72 kB at 284 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/jdom/jdom2/2.0.6/jdom2-2.0.6.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/7.3.1/asm-7.3.1.jar (122 kB at 483 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-dependency-tree/3.0.1/maven-dependency-tree-3.0.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/commons-codec/commons-codec/1.13/commons-codec-1.13.jar (344 kB at 1.4 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/vafer/jdependency/2.3.0/jdependency-2.3.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-tree/7.3.1/asm-tree-7.3.1.jar (53 kB at 199 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-util/7.2-beta/asm-util-7.2-beta.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-analysis/7.3.1/asm-analysis-7.3.1.jar (33 kB at 119 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/com/google/guava/guava/19.0/guava-19.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-dependency-tree/3.0.1/maven-dependency-tree-3.0.1.jar (37 kB at 126 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/vafer/jdependency/2.3.0/jdependency-2.3.0.jar (169 kB at 568 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-util/7.2-beta/asm-util-7.2-beta.jar (81 kB at 269 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/jdom/jdom2/2.0.6/jdom2-2.0.6.jar (305 kB at 1.0 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/google/guava/guava/19.0/guava-19.0.jar (2.3 MB at 6.4 MB/s)
[INFO] Replacing main artifact with repackaged archive
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  24.337 s
[INFO] Finished at: 2026-10-14T12:19:52Z
[INFO] ------------------------------------------------------------------------
 ---> Removed intermediate container 536e4b909f47
 ---> 14faa2696d9c
Step 5/6 : EXPOSE 8080
 ---> Running in c25ae0b9e961
 ---> Removed intermediate container c25ae0b9e961
 ---> 0a3619e931f8
Step 6/6 : CMD ["sh", "-c", "java -jar target/*.jar"]o
 ---> Running in e379ce427ef6
 ---> Removed intermediate container e379ce427ef6
 ---> 9adc42ffcd98
Successfully built 9adc42ffcd98
Successfully tagged multi-stage-example:single
root@ip-172-31-36-56:/home/ubuntu/multi-stage-example#
```

Step 4: Create a multi-stage Dockerfile

Create another file:

nano Dockerfile.multi

Add:

```
# Stage 1: Build the application
FROM maven:3.9-eclipse-temurin-21 AS builder

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests
```

```
# Stage 2: Run the application
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]
```

Explanation:

- `AS builder` – Names the first stage, which builds the application.
- The Maven command compiles and packages the source code.
- The second `FROM` starts a separate runtime stage.
- `COPY --from=builder` – Copies the generated JAR from the builder stage.
- The final image contains the runtime environment and application JAR, without the Maven build environment.

Multi-stage builds separate the build environment from the runtime environment, which can reduce the final image size.

Step 5: Build the multi-stage image

```
docker build -f Dockerfile.multi -t multi-stage-example:multi .
```

```
Downloaded from central: https://repo.maven.apache.org/maven2/org/jdom/jdom2/2.0.6/jdom2-2.0.6.jar (305 kB at 753 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/google/guava/guava/19.0/guava-19.0.jar (2.3 MB at 4.9 MB/s)
[INFO] Replacing main artifact with repackaged archive
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  25.273 s
[INFO] Finished at: 2026-10-14T12:24:08Z
[INFO] ------------------------------------------------------------------------
 ---> Removed intermediate container 3b85cd71ea46
 ---> 8c1e009960c4
Step 5/9 : FROM eclipse-temurin:21-jre
21-jre: Pulling from library/eclipse-temurin
550bcd7b9380: Pulling fs layer
5d8dab7db274: Pulling fs layer
9d8d42fbc560: Pulling fs layer
3b4157e3a962: Pulling fs layer
3b4157e3a962: Download complete
550bcd7b9380: Download complete
24722927dd0c: Download complete
5cbbf2e0e8e1: Download complete
5d8dab7db274: Download complete
9d8d42fbc560: Download complete
5d8dab7db274: Pull complete
3b4157e3a962: Pull complete
9d8d42fbc560: Pull complete
550bcd7b9380: Pull complete
Digest: sha256:ccbad9e7cb84c27092b12c40a4143e5733e050db9c3784d84f632dcf423fb2ef
Status: Downloaded newer image for eclipse-temurin:21-jre
 ---> ccbad9e7cb84
Step 6/9 : WORKDIR /app
 ---> Running in ad253617935a
 ---> Removed intermediate container ad253617935a
 ---> 6c9dcd2ea875
Step 7/9 : COPY --from=builder /app/target/*.jar app.jar
 ---> 8ed50851dc27
Step 8/9 : EXPOSE 8080
 ---> Running in f88bed2c8af7
 ---> Removed intermediate container f88bed2c8af7
 ---> eb2c60922d34
Step 9/9 : CMD ["java", "-jar", "app.jar"]
 ---> Running in 2bd53fe60c9e
 ---> Removed intermediate container 2bd53fe60c9e
 ---> 1586db5eaf15
Successfully built 1586db5eaf15
Successfully tagged multi-stage-example:multi
root@ip-172-31-36-56:/home/ubuntu/multi-stage-example#
```

Step 6: Compare image sizes

docker images

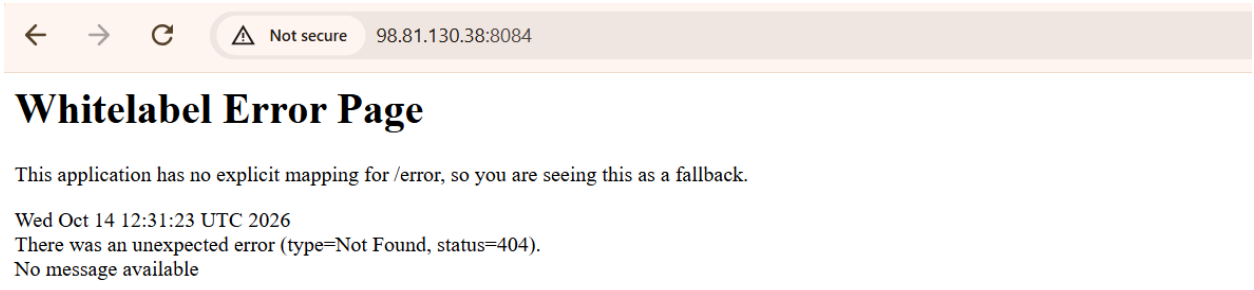
Compare `multi-stage-example:single` and `multi-stage-example:multi`.

The actual sizes depend on the application and base images; the multi-stage image is generally smaller.

```
root@ip-172-31-36-56:/home/ubuntu/multi-stage-example# docker images | grep -E 'multi-stage-example.*(single|multi)'
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
multi-stage-example:multi                                1586db5eaf15       503MB          135MB
multi-stage-example:single                               9adc42ffcd98       988MB          324MB
root@ip-172-31-36-56:/home/ubuntu/multi-stage-example#
```

Step 7: Run the multi-stage application

docker run -d --name multi-app -p 8080:8080 multi-stage-example:multi

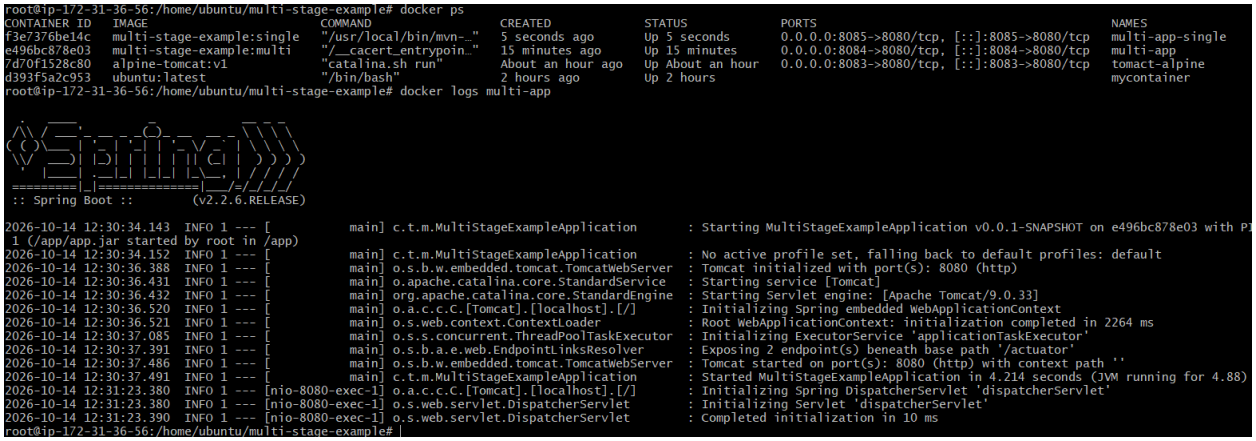


docker run -d --name multi-app-single -p 8085:8080 multi-stage-example:single



Check the container and logs:

docker ps
docker logs multi-app



If the application starts successfully, test it using the endpoint defined by the project.

Conclusion: We built single-stage and multi-stage images and compared their sizes. The multi-stage approach keeps the final runtime image separate from the build tools.

Task 5: Install Docker Compose and Execute a Sample Application

Objective: Install Docker Compose and use it to run a sample multi-container application.

Step 1: Install Docker Compose

On Ubuntu, if Docker Engine and its official package repository are already configured, run:

sudo apt update
sudo apt install docker-compose-v2 -y

Verify the installation:

Sudo docker compose version

```
root@ip-172-31-36-56:/home/ubuntu# docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1~24.04.1
root@ip-172-31-36-56:/home/ubuntu#
```

The Compose plugin is the recommended modern command-line approach for Linux.

Step 2: Create a project directory

Sudo mkdir compose-demo
Sudo cd compose-demo

Step 3: Create a Compose file

Sudo nano compose.yaml

Add the following content:

```
services:
  wordpress:
    image: wordpress:latest
    ports:
      - "8089:80"
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wpPass123
      WORDPRESS_DB_NAME: wordpress
    depends_on:
      - db

  db:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wpPass123
      MYSQL_ROOT_PASSWORD: rootPass123
```

```
    volumes:
      - db_data:/var/lib/mysql

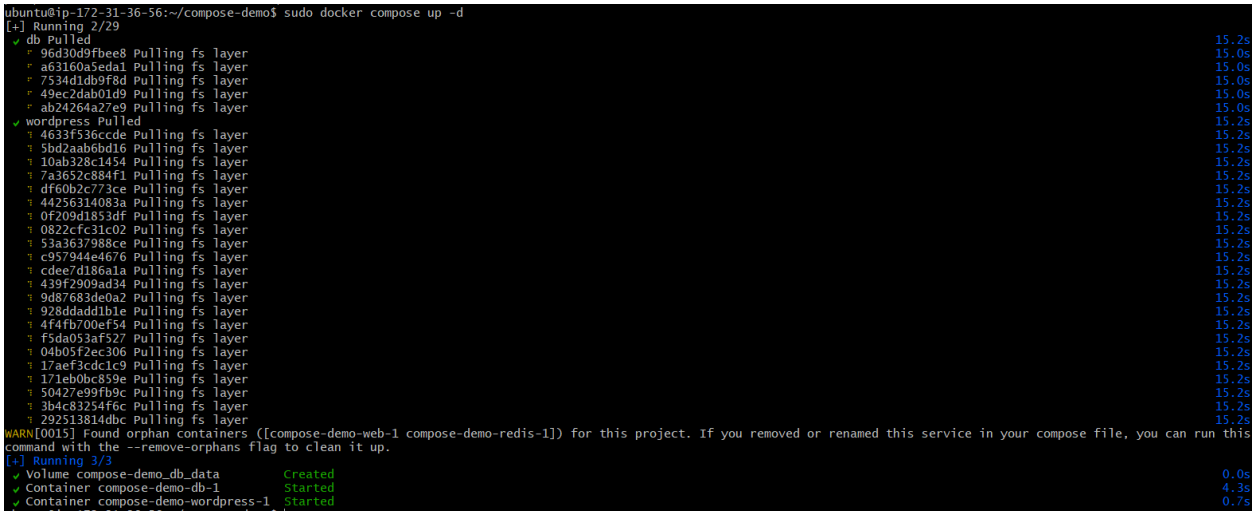
volumes:
  db_data:
```

Explanation:

- `services` – Defines the containers in the application.
- `web` – Runs an Nginx web server.
- `image` – Specifies which image to use.
- `ports` – Maps host port `8080` to container port `80`.
- `redis` – Runs a Redis container.

Step 4: Start the application

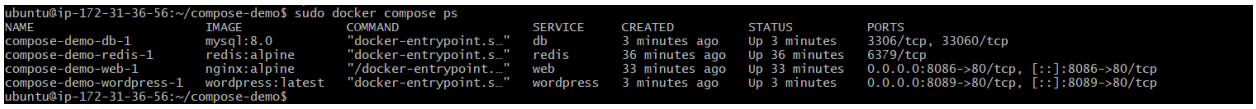
Sudo docker compose up -d



This downloads the images if necessary and starts both services in the background.

Step 5: Verify the containers

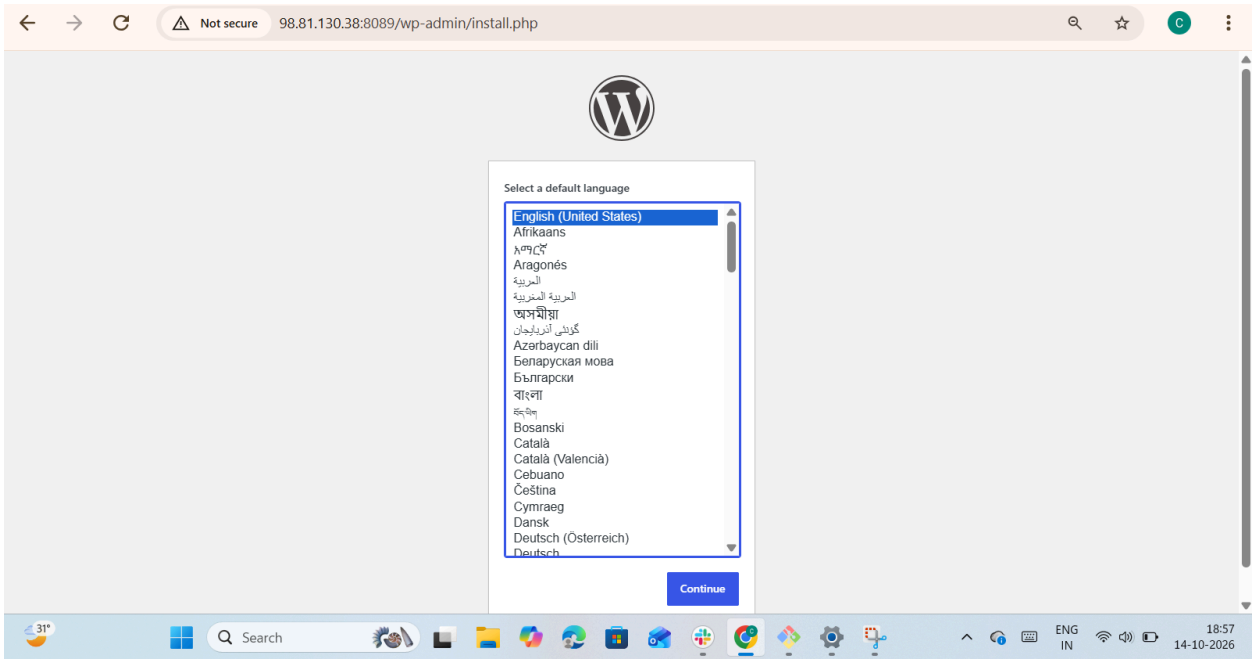
Sudo docker compose ps

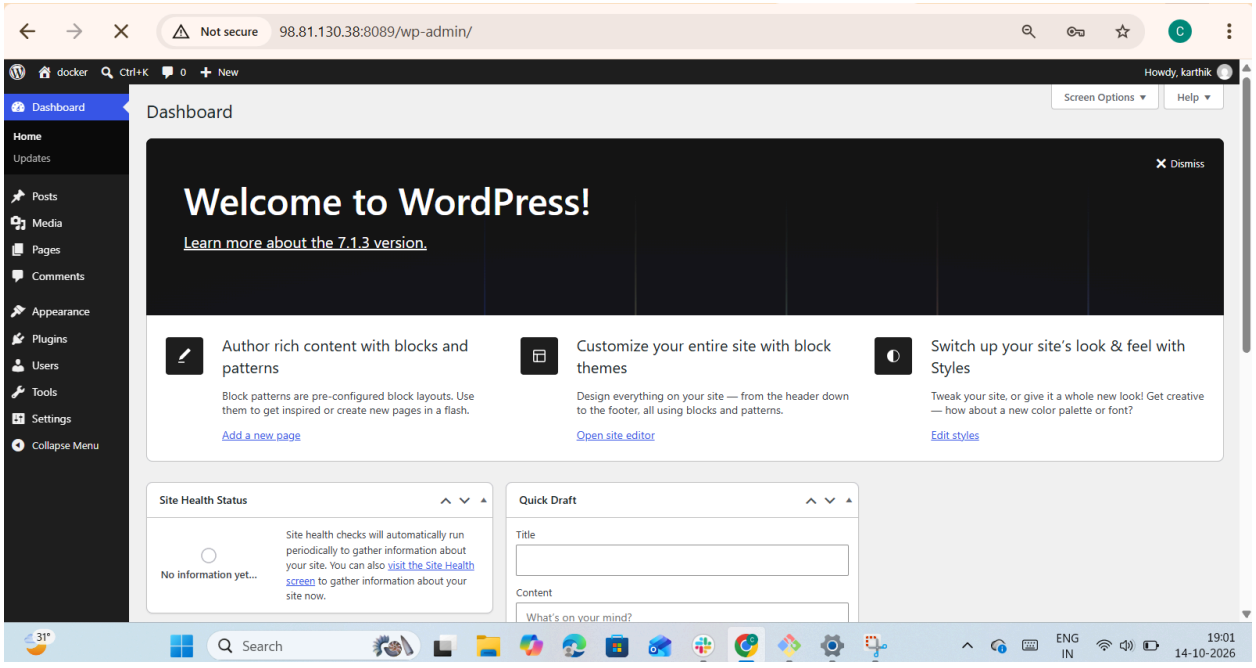


Step 6: Test the Nginx application

Open the following address in your browser:

http://YOUR_SERVER_IP:8089





Step 7: Stop the application

docker compose down

This stops and removes the containers and the Compose network. The downloaded images remain on the system.

Conclusion: We installed Docker Compose and used one YAML file to start and manage Nginx and Redis together.

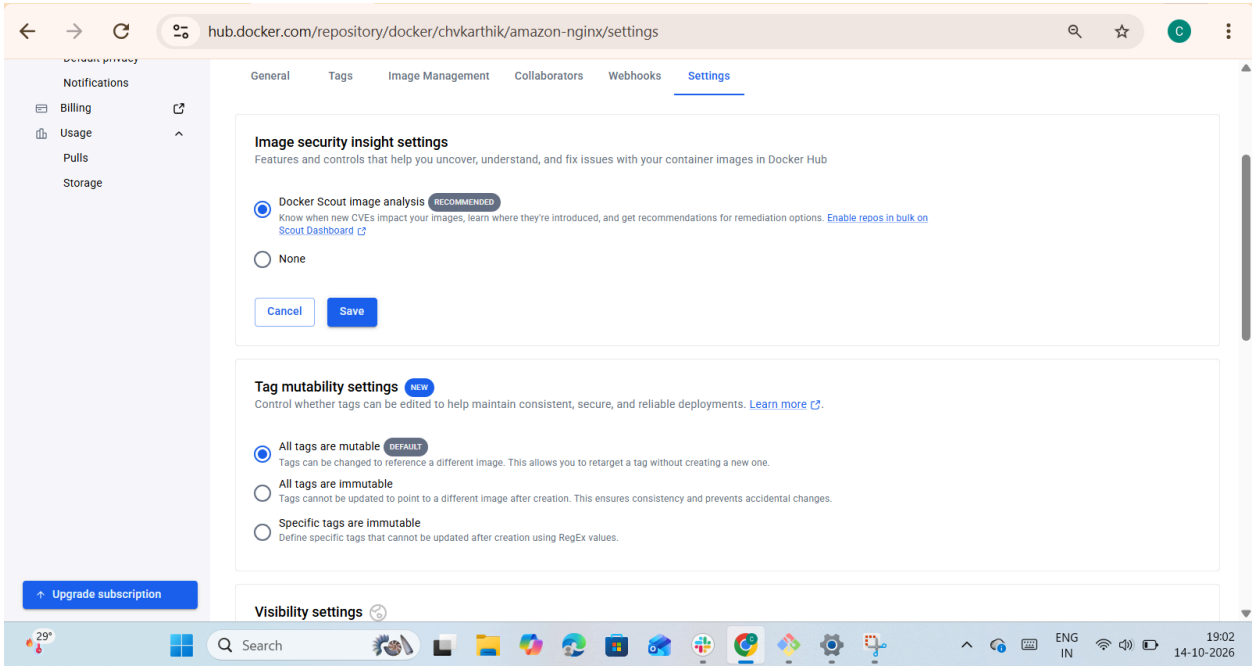
Task 6: Scan Images When Pushed to Docker Hub

Objective: Automatically scan container images for known security vulnerabilities when they are pushed to a Docker registry.

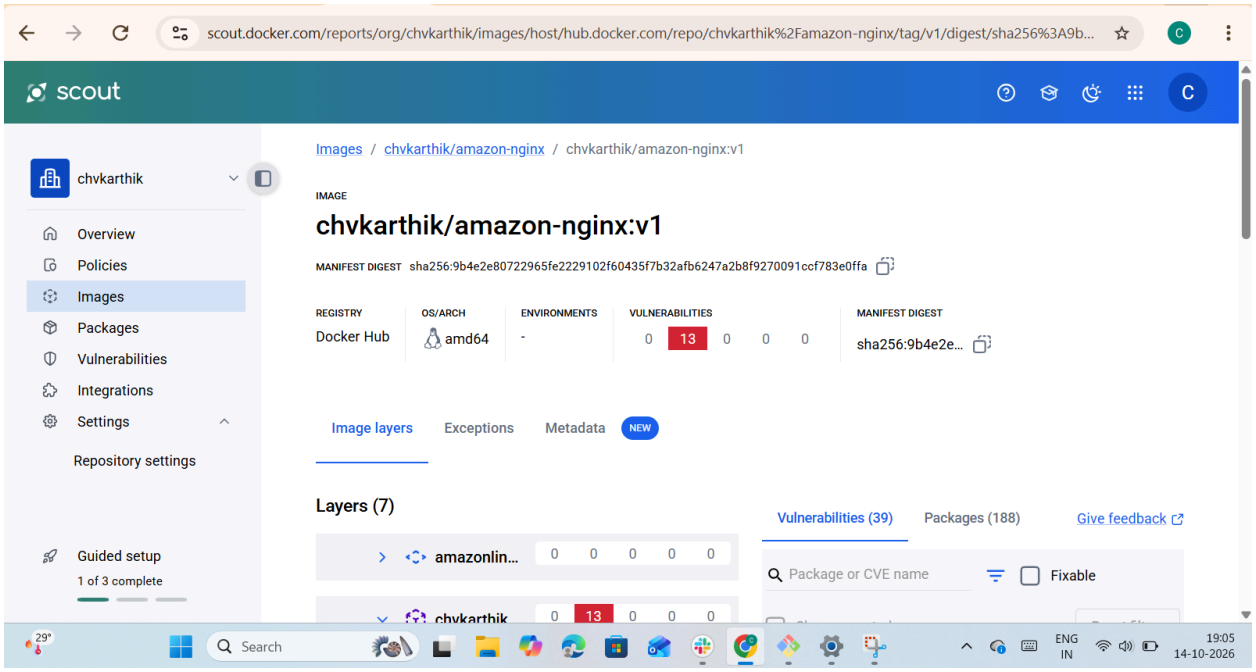
One option is Docker Hub's available image-scanning feature. Availability and configuration depend on the registry plan and settings. Another option is to run an open-source scanner such as Trivy in your build pipeline.

Method A: Configure scanning in Docker Hub

1. Log in to https://hub.docker.com/.
2. Open your repository.
3. Check the repository's security or image-scanning settings.
4. If scan-on-push is available for your account, enable it.



5. Push a new image to the repository.
6. Open the image's security or scan-results section and review the findings.



Method B: Scan an image using Trivy

This method works for scanning images before pushing them to Docker Hub, even if registry-native scanning is unavailable.

Step 1: Install Trivy on Ubuntu

```
sudo apt-get update
sudo apt-get install -y wget apt-transport-https gnupg
```

Add the official Trivy repository and install the scanner:

```
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | \
  gpg --dearmor | sudo tee /usr/share/keyrings/trivy.gpg > /dev/null
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg]
https://aquasecurity.github.io/trivy-repo/deb generic main" | \
  sudo tee /etc/apt/sources.list.d/trivy.list
sudo apt-get update
sudo apt-get install -y trivy
```

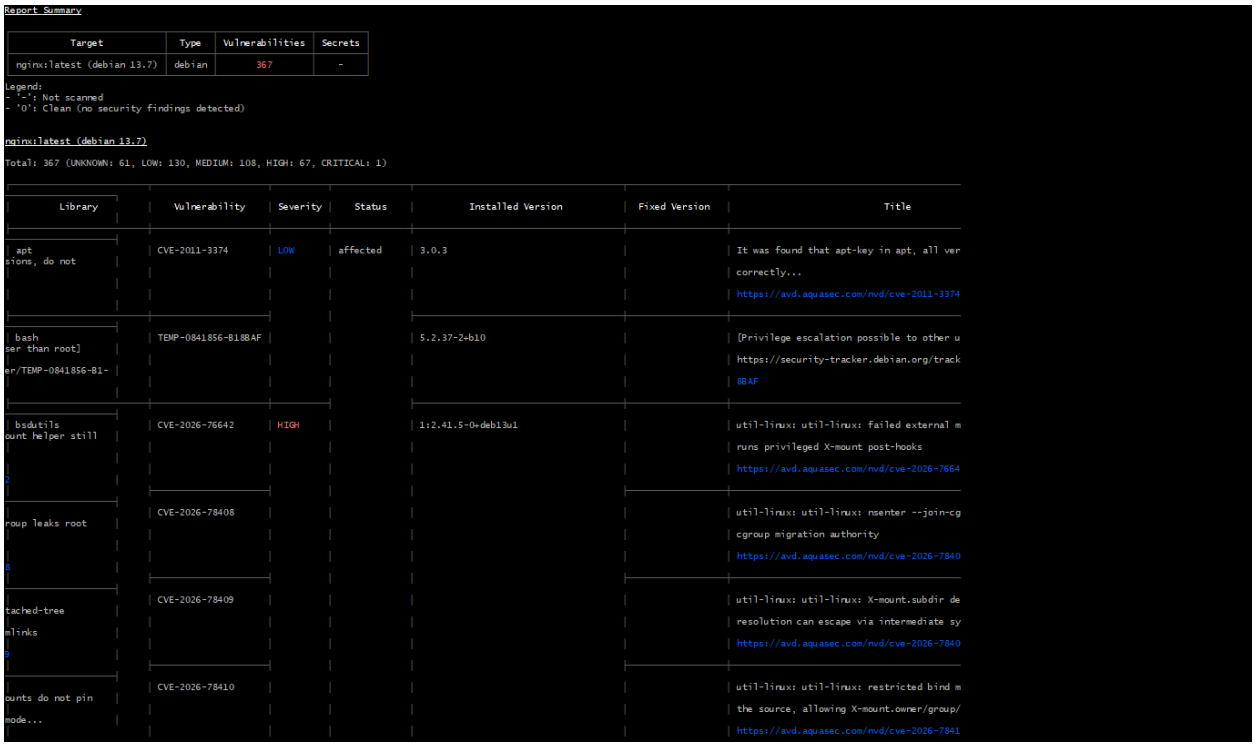
Step 2: Scan a Docker image

```
trivy image nginx:latest
```

Trivy examines the image for known vulnerabilities and reports the findings.

Step 3: Scan an image and fail on serious vulnerabilities

trivy image --exit-code 1 --severity HIGH,CRITICAL nginx:latest



```
Report Summary

┌──────────────────────────┬────────┬─────────────────┬─────────┐
│          Target          │  Type  │ Vulnerabilities │ Secrets │
├──────────────────────────┼────────┼─────────────────┼─────────┤
│ nginx:latest (debian 13.7) │ debian │       367       │    -    │
└──────────────────────────┴────────┴─────────────────┴─────────┘
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

nginx:latest (debian 13.7)

Total: 367 (UNKNOWN: 61, LOW: 130, MEDIUM: 108, HIGH: 67, CRITICAL: 1)

┌──────────┬──────────────────┬──────────┬──────────┬──────────────────┬───────────────┬──────────────────────────────────────────────┐
│ Library  │  Vulnerability   │ Severity │  Status  │ Installed Version │ Fixed Version │                    Title                     │
├──────────┼──────────────────┼──────────┼──────────┼──────────────────┼───────────────┼──────────────────────────────────────────────┤
│ apt      │ CVE-2011-3374    │ LOW      │ affected │ 3.0.3            │               │ It was found that apt-key in apt, all ver    │
│          │                  │          │          │                  │               │ correctly...                                 │
│          │                  │          │          │                  │               │ https://avd.aquasec.com/nvd/cve-2011-3374    │
├──────────┼──────────────────┼──────────┼──────────┼──────────────────┼───────────────┼──────────────────────────────────────────────┤
│ bash     │ TEMP-0841856-B18BAF │       │          │ 5.2.37-2+b10     │               │ [Privilege escalation possible to other u    │
│          │                  │          │          │                  │               │ https://security-tracker.debian.org/track    │
│          │                  │          │          │                  │               │ 8BAF                                         │
├──────────┼──────────────────┼──────────┼──────────┼──────────────────┼───────────────┼──────────────────────────────────────────────┤
│ bsdutils │ CVE-2026-76642   │ HIGH     │          │ 1:2.41.5-0+deb13u1 │             │ util-linux: util-linux: failed external m    │
│          │                  │          │          │                  │               │ runs privileged X-mount post-hooks           │
│          │                  │          │          │                  │               │ https://avd.aquasec.com/nvd/cve-2026-7664    │
│          ├──────────────────┤          │          │                  ├───────────────┼──────────────────────────────────────────────┤
│          │ CVE-2026-78408   │          │          │                  │               │ util-linux: util-linux: nsenter --join-cg    │
│          │                  │          │          │                  │               │ cgroup migration authority                   │
│          │                  │          │          │                  │               │ https://avd.aquasec.com/nvd/cve-2026-7840    │
│          ├──────────────────┤          │          │                  ├───────────────┼──────────────────────────────────────────────┤
│          │ CVE-2026-78409   │          │          │                  │               │ util-linux: util-linux: X-mount.subdir de    │
│          │                  │          │          │                  │               │ resolution can escape via intermediate sy    │
│          │                  │          │          │                  │               │ https://avd.aquasec.com/nvd/cve-2026-7840    │
│          ├──────────────────┤          │          │                  ├───────────────┼──────────────────────────────────────────────┤
│          │ CVE-2026-78410   │          │          │                  │               │ util-linux: util-linux: restricted bind m    │
│          │                  │          │          │                  │               │ the source, allowing X-mount.owner/group/    │
│          │                  │          │          │                  │               │ https://avd.aquasec.com/nvd/cve-2026-7841    │
```

Explanation:

- `trivy image` – Scans a container image.
- `--severity HIGH,CRITICAL` – Focuses on high- and critical-severity findings.
- `--exit-code 1` – Returns a failure exit code when matching vulnerabilities are found.

Conclusion: Docker Hub scanning can provide registry-side results when supported. Trivy provides an additional vulnerability check that can be integrated into a build pipeline. A Trivy command run manually is not itself an automatic scan-on-push trigger.

Task 7: Scan Images When Pushed to AWS ECR

Objective: Configure Amazon Elastic Container Registry (ECR) to scan images for vulnerabilities when they are pushed.

Amazon ECR supports scan-on-push configuration. Basic scanning can identify operating-system package vulnerabilities, while enhanced scanning provides additional capabilities through Amazon Inspector.

Step 1: Install and configure AWS CLI

Check whether AWS CLI is installed:

aws --version

If necessary, install AWS CLI using the official AWS installation instructions.

Configure credentials if you are not using an attached EC2 IAM role:

aws configure

Enter your AWS access key, secret key, default region, and output format when prompted.

For an EC2 instance, an appropriately scoped IAM role is generally preferable to storing long-lived credentials.

Step 2: Create an ECR repository with scan-on-push enabled

Replace `YOUR_REGION` with your AWS region.

```
aws ecr create-repository \
  --repository-name myapp \
  --image-scanning-configuration scanOnPush=true \
  --region YOUR_REGION
```

```
ubuntu@ip-172-31-36-56:~$ aws ecr create-repository \
  --repository-name myapp \
  --image-scanning-configuration scanOnPush=true \
  --region us-east-1
{
    "repository": {
        "repositoryArn": "arn:aws:ecr:us-east-1:686194638896:repository/myapp",
        "registryId": "686194638896",
        "repositoryName": "myapp",
        "repositoryUri": "686194638896.dkr.ecr.us-east-1.amazonaws.com/myapp",
        "createdAt": "2026-10-14T13:47:51.397000+00:00",
        "imageTagMutability": "MUTABLE",
        "imageScanningConfiguration": {
            "scanOnPush": true
        },
        "encryptionConfiguration": {
            "encryptionType": "AES256"
        }
    }
}
```

This creates a repository named `myapp` with basic image scanning enabled on push.

If the repository already exists, enable scanning with:

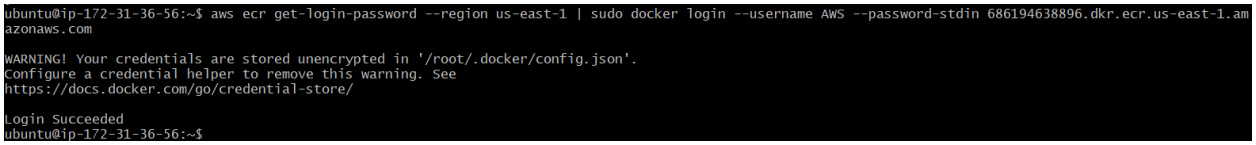
```
aws ecr put-image-scanning-configuration \
  --repository-name myapp \
  --image-scanning-configuration scanOnPush=true \
```

--region YOUR_REGION

Step 3: Log in to Amazon ECR

Replace `YOUR_ACCOUNT_ID` and `YOUR_REGION` with your AWS account ID and region.

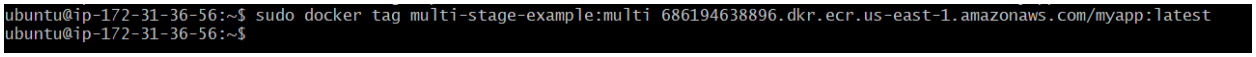
```
aws ecr get-login-password --region YOUR_REGION | \
docker login --username AWS --password-stdin \
YOUR_ACCOUNT_ID.dkr.ecr.YOUR_REGION.amazonaws.com
```



Step 4: Tag the Docker image

For example, use an existing local image named `myapp:latest`:

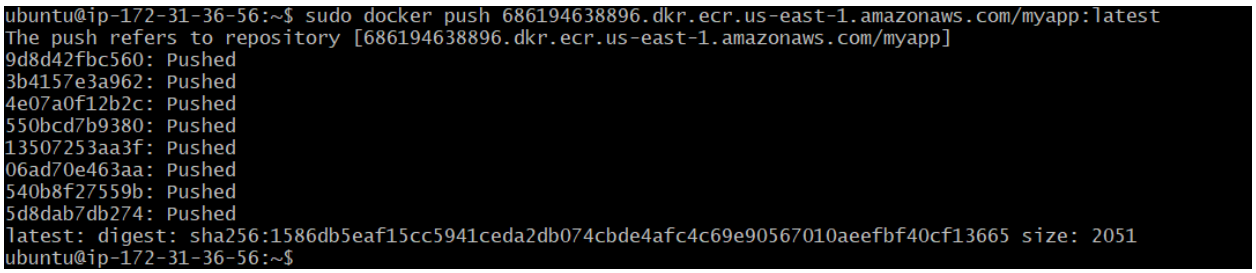
```
docker tag myapp:latest \
YOUR_ACCOUNT_ID.dkr.ecr.YOUR_REGION.amazonaws.com/myapp:latest
```



Step 5: Push the image to ECR

```
docker push \
YOUR_ACCOUNT_ID.dkr.ecr.YOUR_REGION.amazonaws.com/myapp:latest
```

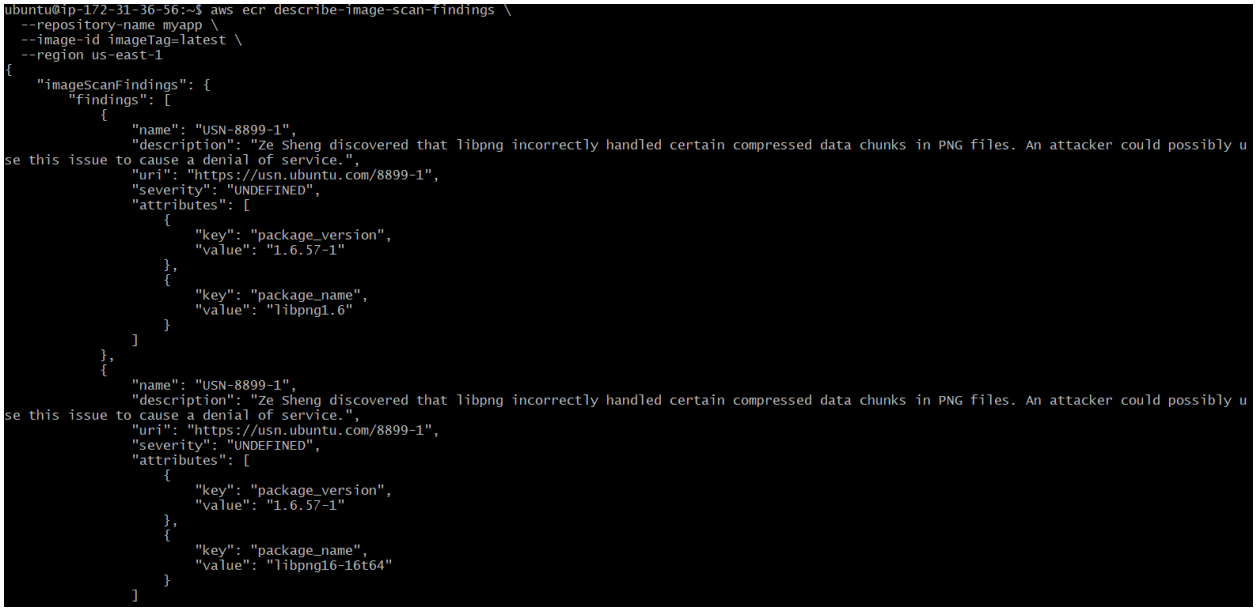
After the push, ECR starts the configured scan.



Step 6: Check the scan findings

```
aws ecr describe-image-scan-findings \
  --repository-name myapp \
  --image-id imageTag=latest \
  --region YOUR_REGION
```

This command displays the scan status and available findings. If the scan is still running, wait and check again.

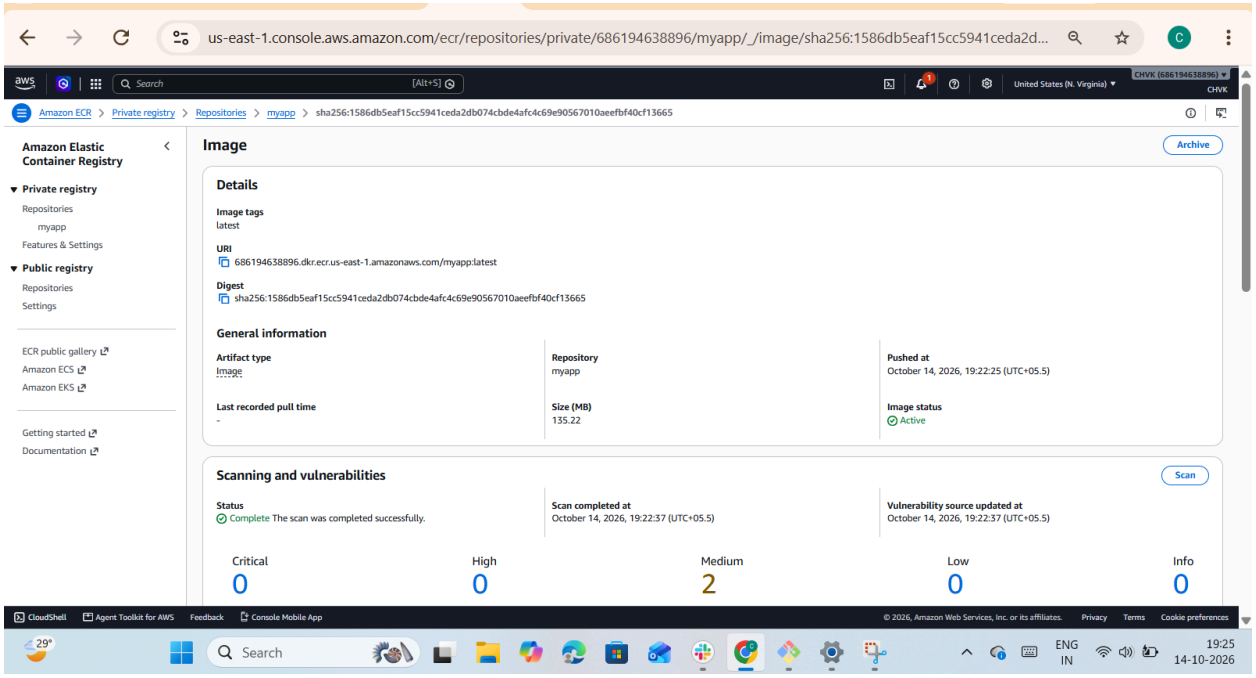


```
ubuntu@ip-172-31-36-56:~$ aws ecr describe-image-scan-findings \
  --repository-name myapp \
  --image-id imageTag=latest \
  --region us-east-1
{
    "imageScanFindings": {
        "findings": [
            {
                "name": "USN-8899-1",
                "description": "Ze Sheng discovered that libpng incorrectly handled certain compressed data chunks in PNG files. An attacker could possibly u
se this issue to cause a denial of service.",
                "uri": "https://usn.ubuntu.com/8899-1",
                "severity": "UNDEFINED",
                "attributes": [
                    {
                        "key": "package_version",
                        "value": "1.6.57-1"
                    },
                    {
                        "key": "package_name",
                        "value": "libpng1.6"
                    }
                ]
            },
            {
                "name": "USN-8899-1",
                "description": "Ze Sheng discovered that libpng incorrectly handled certain compressed data chunks in PNG files. An attacker could possibly u
se this issue to cause a denial of service.",
                "uri": "https://usn.ubuntu.com/8899-1",
                "severity": "UNDEFINED",
                "attributes": [
                    {
                        "key": "package_version",
                        "value": "1.6.57-1"
                    },
                    {
                        "key": "package_name",
                        "value": "libpng16-16t64"
                    }
                ]
```

You can also view the repository in the AWS Console:

1. Open Amazon ECR.
2. Select the correct AWS region.
3. Open Repositories and select `myapp`.
4. Select the image.
5. Review the image-scanning findings.

Conclusion: We configured AWS ECR to scan images when they are pushed and used the AWS CLI to inspect vulnerability findings



.

Task 8: Create a Jenkins Pipeline to Build a Docker Image and Push It to Docker Hub

Objective: Automate Docker image creation and publishing using a Jenkins Pipeline.

Step 1: Prepare the project

Your project should contain at least a Dockerfile and the application files required to build the image.

For example, create a simple project directory:

```
mkdir jenkins-docker-app
cd jenkins-docker-app
```

Create a file named `Dockerfile`:

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

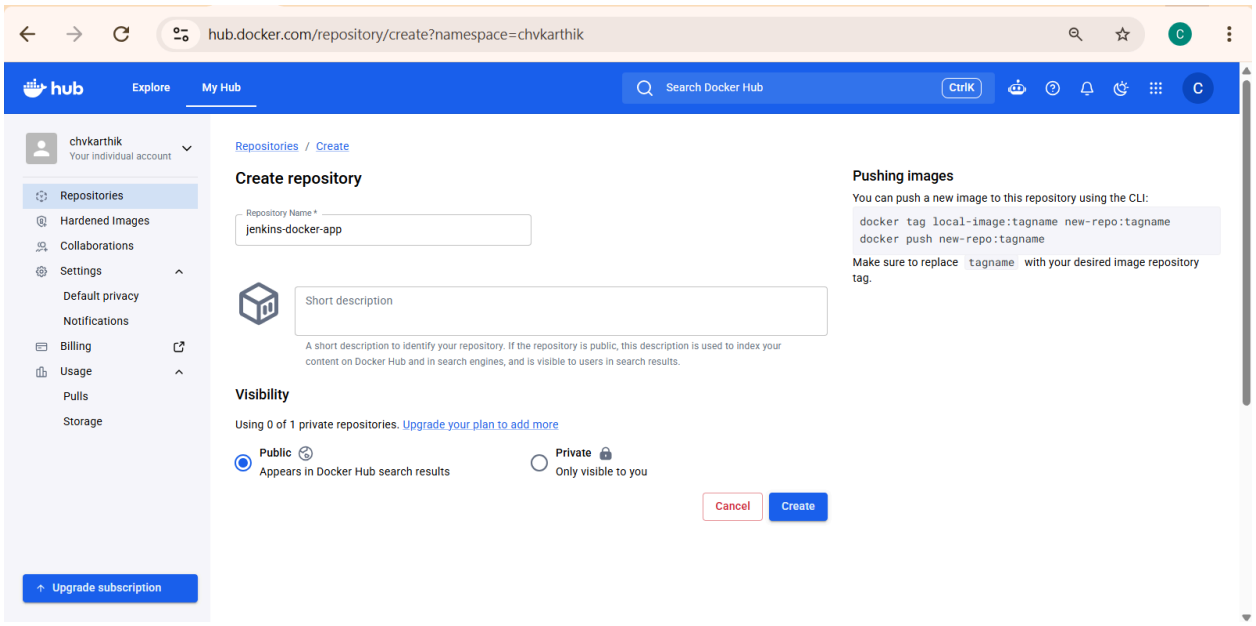
EXPOSE 80
```

Create the HTML file:

```
echo '<h1>Hello from Jenkins Docker Pipeline!</h1>' > index.html
```

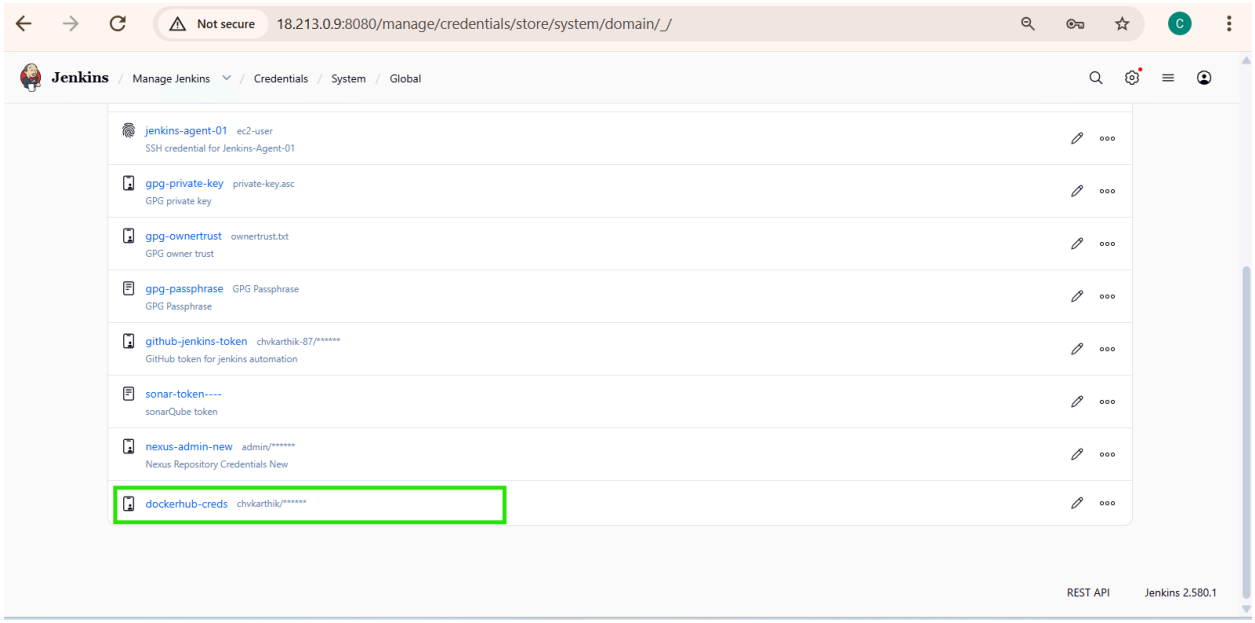
Upload or commit these files to the Git repository that Jenkins will use.

After that create a repository in dockerhub



Step 2: Configure Docker Hub credentials in Jenkins

1. Open Jenkins in your browser.
2. Go to Manage Jenkins.
3. Select Credentials.
4. Open the appropriate credentials store and domain.
5. Select Add Credentials.
6. Choose Username with password.
7. Enter your Docker Hub username.
8. Enter a Docker Hub access token as the password.
9. Set the ID to `dockerhub-creds`.
10. Save the credentials.



Use a Docker Hub access token rather than your account password.

Step 3: Ensure Jenkins can run Docker commands

The Jenkins agent that executes the pipeline must have Docker installed and access to the Docker daemon. If using a Linux agent, verify that the Jenkins user has the necessary permissions.

Run on the agent:

```
docker --version
docker ps
```

If these commands fail, configure Docker access before running the pipeline.

Step 4: Create a Jenkins Pipeline job

1. Open the Jenkins dashboard.
2. Select New Item.
3. Enter the job name `docker-hub-pipeline`.
4. Select Pipeline.
5. Click OK.
6. Scroll to the Pipeline section.
7. Under Definition, select Pipeline script.

Step 5: Add the pipeline script

Paste the following script into the Pipeline script editor.

Replace `YOUR_DOCKERHUB_USERNAME` with your Docker Hub username.

```
pipeline {
    agent any
    environment {
        DOCKER_IMAGE = 'chvkarthik/jenkins-docker-app'
        APP_DIR = '/opt/jenkins-docker-app'
    }
    stages {
        stage('Checkout') {
            steps {
                echo 'Using application files from Jenkins server'
                sh 'ls -l ${APP_DIR}'
            }
        }
        stage('Build Docker Image') {
            steps {
                sh '''
                    docker build \
                    -t DOCKER_IMAGE:{BUILD_NUMBER} \
                    ${APP_DIR}
                '''
            }
```

```
        }
        stage('Login to Docker Hub') {
            steps {
                withCredentials([
                    usernamePassword(
                        credentialsId: 'dockerhub-creds',
                        usernameVariable: 'DOCKERHUB_USER',
                        passwordVariable: 'DOCKERHUB_TOKEN'
                    )
                ]) {
                    sh '''
                        set +x
                        echo "$DOCKERHUB_TOKEN" |
                        docker login \
                        -u "$DOCKERHUB_USER" \
                        --password-stdin
                    '''
                }
            }
        }
        stage('Push Docker Image') {
            steps {
                sh 'docker push DOCKER_IMAGE:{BUILD_NUMBER}'
```

```
        }
    }
}
post {
    always {
        sh 'docker logout || true'
    }
}
}
```

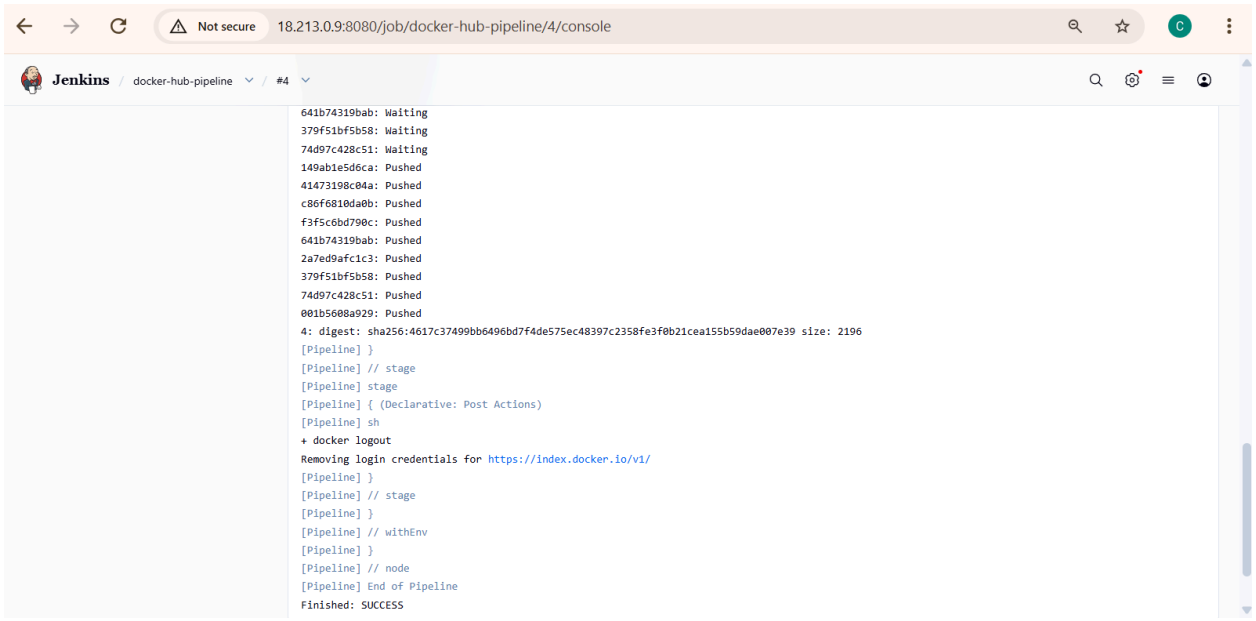
Pipeline explanation:

- `Checkout` – Checks out the configured source code.
- `Build Docker Image` – Builds an image from the Dockerfile.
- `Login to Docker Hub` – Authenticates using the Jenkins credentials store.
- `Push Docker Image` – Pushes the tagged image to Docker Hub.
- `BUILD_NUMBER` – Gives each Jenkins build a unique tag within that job.
- `post` – Logs out of Docker Hub after the pipeline finishes.

Important: Configure the job's source code repository under its SCM settings, or use an appropriate Pipeline-from-SCM configuration. The `checkout scm` step requires a configured SCM source.

Step 6: Run the pipeline

1. Save the job configuration.
2. Select Build Now.
3. Open the running build.
4. Select Console Output.
5. Check that all four stages complete successfully.

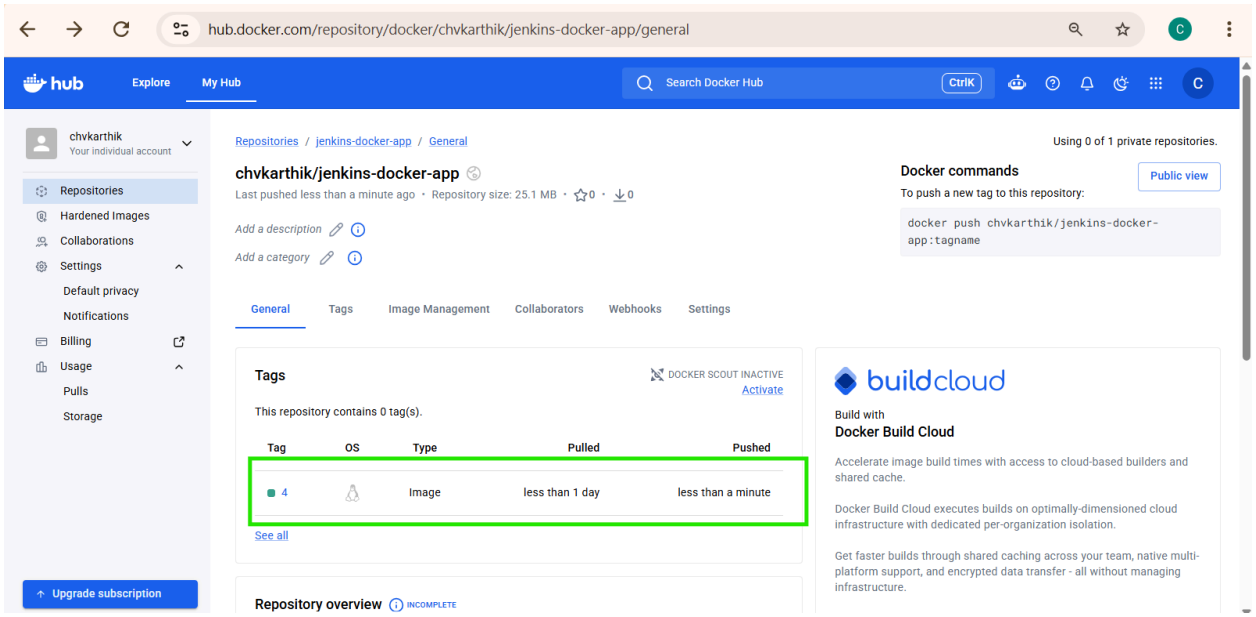


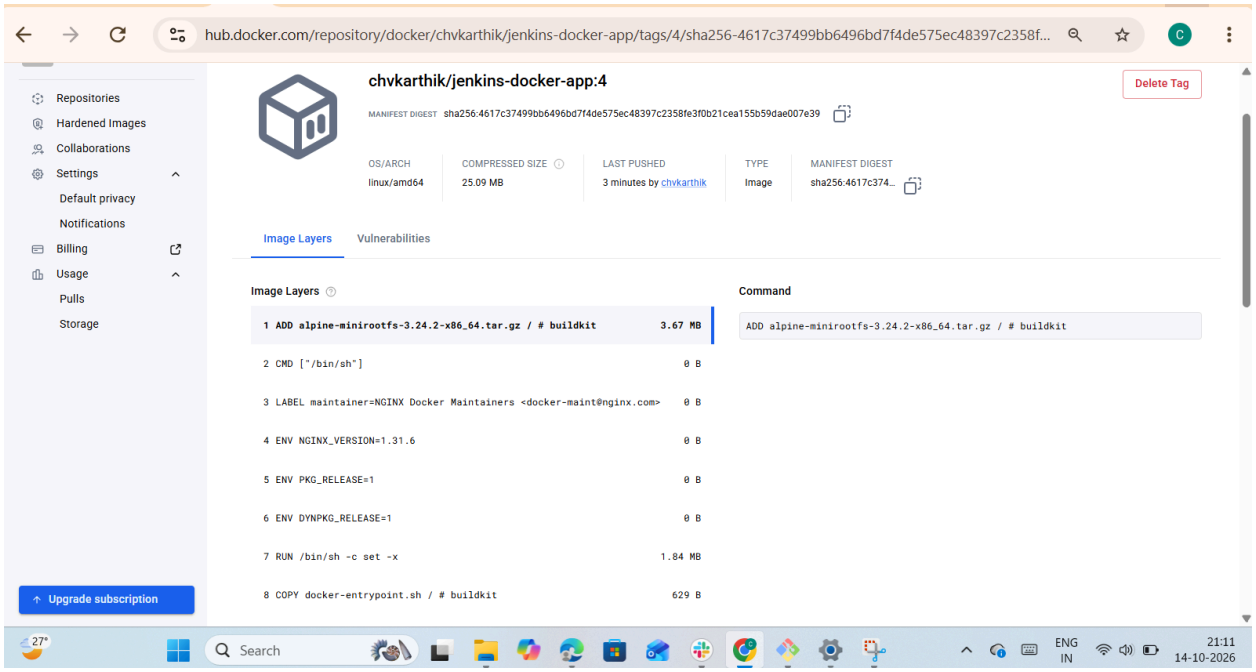
Step 7: Verify the Docker Hub image

Open https://hub.docker.com/, select your repository, and verify that an image tagged with the Jenkins build number has been pushed.

You can also verify from the Docker agent:

docker images





Conclusion: We created a Jenkins Pipeline that checks out source code, builds a Docker image, authenticates securely, and pushes the image to Docker Hub.

Final Assignment Summary

Task	Implementation	Verification
1	Create an image from a container	`docker images`
2	Save, copy, and load an image	`docker images` on the server
3	Build Alpine with Tomcat	Open port 8080
4	Build single-stage and multi-stage images	`docker images`

5	Run an application with Compose	`docker compose ps`
6	Configure registry scanning / Trivy	Review scan findings
7	Enable ECR scan-on-push	ECR scan findings
8	Build and push using Jenkins	Jenkins Console Output and Docker Hub

Final Conclusion

This assignment covers Docker image management, image customization, build optimization, multi-container applications, security scanning, AWS ECR, and CI/CD automation using Jenkins.